

Homologação PagBank — Integração Ozoxx Experience

Documentação técnica de requests e respostas utilizados em produção

Empresa	Ozoxx Experience
Domínio de produção	ozoxxexperience.com.br
URL base do checkout	https://ozoxxexperience.com.br
URL de notificação (webhook)	https://ozoxxexperience.com.br/api/webhook/pagbank
Endpoint base PagBank	https://api.pagseguro.com
Versão da API utilizada	v4
Métodos suportados	PIX e Cartão de Crédito (Checkout hospedado)
Linguagem do backend	Python 3.11 / FastAPI
Biblioteca HTTP	httpx

Visão geral da integração

O sistema da Ozoxx Experience utiliza a API v4 da PagBank em dois fluxos distintos:

1) Fluxo PIX: chamada direta a `POST /orders` com bloco `qr_codes`, gerando o QR Code estático que o participante paga diretamente em seu app bancário.

2) Fluxo Cartão de Crédito: chamada a `POST /checkouts` que retorna um `payment_link` para o ambiente hospedado da PagBank (parcelamento em até 12x). Após o pagamento, o cliente é redirecionado de volta para o site.

Em ambos os fluxos, recebemos notificações no endpoint `POST`

`https://ozoxxexperience.com.br/api/webhook/pagbank` e também fazemos *polling* de segurança a cada 60s consultando `GET /orders/{id}` e `GET /checkouts/{id}`.

Endpoints PagBank utilizados

Endpoint	Uso
<code>POST /public-keys</code>	Health-check da conexão e validação do token (botão 'Testar conexão' do admin)
<code>POST /orders</code>	Criação de pedido com PIX (gera QR Code)
<code>POST /checkouts</code>	Criação de checkout hospedado (PIX + Cartão de Crédito)
<code>GET /orders/{id}</code>	Consulta do status do pedido (polling de segurança)
<code>GET /checkouts/{id}</code>	Consulta do status do checkout e seus pedidos vinculados
Webhook entrante	Recebimento de notificações da PagBank em <code>https://ozoxxexperience.com.br/api/webhook/pag</code>

1) Health-check — POST /public-keys

Utilizado pelo botão *Testar conexão* no painel administrativo. Permite validar que o token informado é aceito pela PagBank antes de salvar nas configurações.

Request

```
POST https://api.pagseguro.com/public-keys
Authorization: Bearer {TOKEN_PAGBANK}
Content-Type: application/json
Accept: application/json
```

```
{
  "type": "card"
}
```

Response (200 OK)

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{
  "public_key": "-----BEGIN PUBLIC KEY-----\nMIIBIjANBgkqhkiG9w0BAQEFAAOCAQY\n-----END PUBLIC KEY-----",
  "created_at": "2026-06-10T14:23:11.000-03:00",
  "expires_at": "2027-06-10T14:23:11.000-03:00"
}
```

2) PIX — POST /orders

Cria um pedido com QR Code PIX. O valor é enviado em **centavos**. Cada pedido tem um `reference_id` único gerado pelo nosso sistema (formato `ord_xxxxxxxxxxxx`).

Request

POST `https://api.pagseguro.com/orders`
Authorization: Bearer `{TOKEN_PAGBANK}`
Content-Type: `application/json`
Accept: `application/json`

```
{
  "reference_id": "ord_5fcd5da8b3e2",
  "customer": {
    "name": "João da Silva",
    "email": "joao.silva@email.com.br",
    "tax_id": "12345678909",
    "phones": [
      {
        "country": "55",
        "area": "11",
        "number": "999998888",
        "type": "MOBILE"
      }
    ]
  },
  "items": [
    {
      "reference_id": "ord_5fcd5da8b3e2",
      "name": "Passaporte Ozox (1x) – Ozox Experience",
      "quantity": 1,
      "unit_amount": 220000
    }
  ],
  "qr_codes": [
    { "amount": { "value": 220000 } }
  ],
  "notification_urls": [
    "https://ozoxexperience.com.br/api/webhook/pagbank"
  ]
}
```

Response (201 Created)

HTTP/1.1 201 Created
Content-Type: `application/json`

```
{
  "id": "ORDE_8B3D9C7E-12AB-4F56-9D8E-1A2B3C4D5E6F",
  "reference_id": "ord_5fcd5da8b3e2",
  "created_at": "2026-06-10T14:32:11.000-03:00",
  "customer": {
    "name": "João da Silva",
    "email": "joao.silva@email.com.br",
    "tax_id": "12345678909"
  },
  "items": [
    {
      "reference_id": "ord_5fcd5da8b3e2",
      "name": "Passaporte Ozox (1x) – Ozox Experience",
      "quantity": 1,
      "unit_amount": 220000
    }
  ],
}
```

```

"qr_codes": [
  {
    "id": "QRCO_F1E2D3C4-AB12-CD34-EF56-789012345678",
    "expiration_date": "2026-06-11T14:32:11.000-03:00",
    "amount": { "value": 220000 },
    "text": "00020126360014BR.GOV.BCB.PIX0114+5511999998888...6304ABCD",
    "links": [
      {
        "rel": "QRCODE.PNG",
        "href": "https://api.pagseguro.com/qrcode/QRCO_F1E2D3C4.../png",
        "media": "image/png",
        "type": "image/png"
      }
    ]
  }
],
"notification_urls": [
  "https://ozoxxexperience.com.br/api/webhook/pagbank"
]
}

```

Como utilizamos o response

Persistimos no nosso banco de dados: `pagbank_order_id` = `id`, `pagbank_qr_code_url` = `links[0].href`, `pagbank_qr_code_text` = `qr_codes[0].text`. O QR é então renderizado na nossa página de pagamento para o participante.

3) Cartão de Crédito — POST /checkouts

Para pagamento com cartão utilizamos o **Checkout hospedado** da PagBank. Enviamos o pedido para a PagBank e recebemos um `payment_link` para o qual o participante é redirecionado. Após o pagamento, a PagBank redireciona o cliente de volta para `https://ozoxxexperience.com.br/payment/{order_id}`.

Request

POST `https://api.pagseguro.com/checkouts`

Authorization: Bearer {TOKEN_PAGBANK}

Content-Type: application/json

Accept: application/json

```
{
  "reference_id": "ord_5fcd5da8b3e2",
  "customer": {
    "name": "João da Silva",
    "email": "joao.silva@email.com.br",
    "tax_id": "12345678909",
    "phones": [
      {
        "country": "55",
        "area": "11",
        "number": "999998888",
        "type": "MOBILE"
      }
    ]
  },
  "items": [
    {
      "reference_id": "ord_5fcd5da8b3e2",
      "name": "Passaporte Ozox (1x) – Ozox Experience",
      "quantity": 1,
      "unit_amount": 220000
    }
  ],
  "payment_methods": [
    { "type": "CREDIT_CARD" },
    { "type": "PIX" }
  ],
  "payment_methods_configs": [
    {
      "type": "CREDIT_CARD",
      "config_options": [
        { "option": "INSTALLMENTS_LIMIT", "value": "12" }
      ]
    }
  ],
  "redirect_url": "https://ozoxxexperience.com.br/payment/ord_5fcd5da8b3e2",
  "notification_urls": [
    "https://ozoxxexperience.com.br/api/webhook/pagbank"
  ],
  "payment_notification_urls": [
    "https://ozoxxexperience.com.br/api/webhook/pagbank"
  ]
}
```

Response (201 Created)

HTTP/1.1 201 Created

Content-Type: application/json

```
{
  "id": "CHEC_658759E5-86B9-4D80-8AC5-5FC01C5AB432",
  "reference_id": "ord_5fcd5da8b3e2",
```

```

"created_at": "2026-06-10T14:35:11.000-03:00",
"expiration_date": "2026-06-11T14:35:11.000-03:00",
"status": "ACTIVE",
"customer": {
  "name": "João da Silva",
  "email": "joao.silva@email.com.br",
  "tax_id": "12345678909"
},
"items": [
  {
    "reference_id": "ord_5fcd5da8b3e2",
    "name": "Passaporte Ozoxx (1x) – Ozoxx Experience",
    "quantity": 1,
    "unit_amount": 220000
  }
],
"payment_methods": [
  { "type": "CREDIT_CARD" },
  { "type": "PIX" }
],
"links": [
  {
    "rel": "PAY",
    "href": "https://pagamento.pagbank.com.br/pagamento?code=edbe2319-beec-4700-b267-a2c291b4d27b",
    "media": "text/html",
    "type": "text/html",
    "method": "GET"
  },
  {
    "rel": "SELF",
    "href": "https://api.pagueseguro.com/checkouts/CHEC_658759E5-86B9-4D80-8AC5-5FC01C5AB432",
    "media": "application/json",
    "type": "application/json",
    "method": "GET"
  }
],
"origin": "CHECKOUT_WEB"
}

```

Como utilizamos o response

Procuramos no array links o item com rel: PAY e armazenamos como `pagbank_payment_link`. O participante é então redirecionado via `window.location.href` para esta URL.

4) Consulta de pedido — GET /orders/{id}

Utilizado tanto pelo nosso job de *autopoll* (a cada 60 segundos) quanto pelo endpoint *refresh-status* chamado pela página de pagamento do cliente. Confirma o status real do pedido sem depender exclusivamente do webhook.

Request

```
GET https://api.pagseguro.com/orders/ORDE_8B3D9C7E-12AB-4F56-9D8E-1A2B3C4D5E6F
Authorization: Bearer {TOKEN_PAGBANK}
Accept: application/json
```

Response (200 OK)

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{
  "id": "ORDE_8B3D9C7E-12AB-4F56-9D8E-1A2B3C4D5E6F",
  "reference_id": "ord_5fcd5da8b3e2",
  "status": "PAID",
  "created_at": "2026-06-10T14:32:11.000-03:00",
  "customer": {
    "name": "João da Silva",
    "email": "joao.silva@email.com.br",
    "tax_id": "12345678909"
  },
  "charges": [
    {
      "id": "CHAR_F43FE039-9625-459D-873B-214339BC405C",
      "reference_id": "ord_5fcd5da8b3e2",
      "status": "PAID",
      "created_at": "2026-06-10T14:34:02.000-03:00",
      "paid_at": "2026-06-10T14:34:02.000-03:00",
      "amount": { "value": 220000, "currency": "BRL" },
      "payment_response": {
        "code": "200000",
        "message": "SUCESSO"
      },
      "payment_method": { "type": "PIX" }
    }
  ]
}
```

Status possíveis em `charges[].status`

status	significado
WAITING	Aguardando pagamento (recém-criado)
IN_ANALYSIS	Pagamento em análise pela PagBank
PAID	Pagamento aprovado e capturado
DECLINED	Pagamento recusado pelo emissor
CANCELED	Pedido cancelado

5) Consulta de checkout — GET /checkouts/{id}

Após o pagamento via cartão de crédito no ambiente hospedado, consultamos o checkout para obter os pedidos vinculados (orders[]) e em seguida consultamos cada um deles via GET /orders/{id} para verificar se o pagamento foi efetivamente aprovado.

Request

```
GET https://api.pagseguro.com/checkouts/CHEC_658759E5-86B9-4D80-8AC5-5FC01C5AB432
Authorization: Bearer {TOKEN_PAGBANK}
Accept: application/json
```

Response (200 OK)

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{
  "id": "CHEC_658759E5-86B9-4D80-8AC5-5FC01C5AB432",
  "reference_id": "ord_5fcd5da8b3e2",
  "created_at": "2026-06-10T14:35:11.000-03:00",
  "status": "ACTIVE",
  "customer": {
    "name": "João da Silva",
    "email": "joao.silva@email.com.br",
    "tax_id": "12345678909"
  },
  "orders": [
    {
      "id": "ORDE_B81C6E09-68EC-4354-83F7-7F78CCE6CAE0",
      "links": [
        {
          "rel": "SELF",
          "href": "https://api.pagseguro.com/orders/ORDE_B81C6E09-68EC-4354-83F7-7F78CCE6CAE0",
          "method": "GET"
        }
      ]
    }
  ],
  "links": [
    {
      "rel": "PAY",
      "href": "https://pagamento.pagbank.com.br/pagamento?code=edbe2319-...",
      "method": "GET"
    }
  ],
  "origin": "CHECKOUT_WEB"
}
```

6) Webhook recebido — POST

<https://ozoxxexperience.com.br/api/webhook/pagbank>

A PagBank notifica o nosso sistema sempre que há mudança de status em um pedido ou checkout. Nosso endpoint aceita o payload, identifica o pedido localmente (por `pagbank_order_id`, `pagbank_checkout_id` ou `reference_id`), e em seguida re-consulta a PagBank via GET `/orders/{id}` para confirmar o status real antes de atualizar.

Payload entrante (exemplo)

```
POST /api/webhook/pagbank HTTP/1.1
Host: ozoxxexperience.com.br
Content-Type: application/json
```

```
{
  "id": "ORDE_8B3D9C7E-12AB-4F56-9D8E-1A2B3C4D5E6F",
  "reference_id": "ord_5fcd5da8b3e2",
  "charges": [
    {
      "id": "CHAR_F43FE039-9625-459D-873B-214339BC405C",
      "status": "PAID",
      "paid_at": "2026-06-10T14:34:02.000-03:00",
      "amount": { "value": 220000, "currency": "BRL" }
    }
  ]
}
```

Resposta enviada ao PagBank

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{
  "ok": true,
  "status": "PAID",
  "order_id": "ord_5fcd5da8b3e2"
}
```

Importante: nosso sistema sempre responde com 2xx para qualquer payload válido, e em paralelo executa o re-fetch via GET para garantir idempotência (não confiamos cegamente no payload do webhook). Se o pedido não for encontrado retornamos 200 com `not_found: true` para evitar retries desnecessários da PagBank.

7) Cenários de erro — Responses não-200

Documentamos a seguir os erros que tratamos no fluxo. Em todos os casos o erro é capturado, logado, persistido no campo `payment_error` do pedido e exibido ao participante na tela de pagamento, com botão para reprocessar a chamada.

7.1) CPF inválido — 400 Bad Request

HTTP/1.1 400 Bad Request
Content-Type: application/json

```
{
  "error_messages": [
    {
      "code": "40002",
      "description": "must be a valid CPF or CNPJ",
      "parameter_name": "customer.tax_id"
    }
  ]
}
```

Tratamento: o usuário recebe a mensagem *'CPF/CNPJ inválido. Verifique e tente novamente'* na tela de pagamento.

7.2) Token inválido — 401 Unauthorized

HTTP/1.1 401 Unauthorized
Content-Type: application/json

```
{
  "error_messages": [
    {
      "code": "40001",
      "description": "invalid_credential"
    }
  ]
}
```

7.3) Pagamento recusado pelo emissor

HTTP/1.1 200 OK
Content-Type: application/json

```
{
  "id": "ORDE_8B3D9C7E-...",
  "charges": [
    {
      "id": "CHAR_...",
      "status": "DECLINED",
      "payment_response": {
        "code": "20003",
        "message": "Transação não autorizada pelo emissor"
      },
      "payment_method": { "type": "CREDIT_CARD" }
    }
  ]
}
```

Tratamento: status do pedido vai para DECLINED, participante é avisado e pode tentar novamente com outro cartão.

8) Apêndice — Stack técnica

Backend: Python 3.11 / FastAPI. Cliente HTTP `httpx.AsyncClient` com timeout de 30 segundos para todas as chamadas. Logs estruturados em `journalctl -u ozoxx-backend`.

Hospedagem: VPS Ubuntu 22.04, nginx 1.18 como proxy reverso, gunicorn com 4 workers `uvicorn.workers.UvicornWorker`.

Persistência: MongoDB 7.0 local. Cada pedido registra: `pagbank_order_id`, `pagbank_checkout_id`, `pagbank_qr_code_url`, `pagbank_qr_code_text`, `pagbank_payment_link`, `status`, `paid_at`, `payment_error`.

Segurança: token PagBank armazenado em coleção segregada do MongoDB, nunca exposto ao frontend. Todas as chamadas são server-side. Comunicação com a PagBank exclusivamente via HTTPS TLS 1.2+.

Redundância de confirmação de pagamento: três camadas complementares — (1) webhook imediato da PagBank, (2) consulta sob demanda quando o usuário recarrega a tela de pagamento, (3) job em background `autopoll` que varre pedidos em estado `WAITING` a cada 60s. Pedidos não pagos em 7 dias são automaticamente cancelados.

Contato técnico

E-mail: experience@ozoxx.com.br

Domínio em produção: <https://ozoxxexperience.com.br>

Webhook configurado: <https://ozoxxexperience.com.br/api/webhook/pagbank>